

UCS503-Software Engineering Project

Academic Year 2026-2027

**THAPAR INSTITUTE OF ENGINEERING &
TECHNOLOGY**

(Deemed to be University), Patiala, Punjab

**TravelSphere: Collaborative Travel
Discovery & Social Platform**

Dynamic Itinerary Sharing, Destination Discovery, and Traveler Social
Network Platform

Submitted by:

1024030781 — Ayush Gupta
1024030762 — Niharika Chawla
1024030862 — Sarwar Singh Dhillon

B.E. Third Year, Computer Engineering

Submitted to:

Dr. Jeelani Asif
Department of Computer Science and
Engineering

Contents

1	Introduction	2
1.1	Background and Context	2
1.2	Problem Statement	2
1.3	Project Objectives	2
2	Methodology and Design	3
2.1	System Architecture	3
2.1.1	High-Level Design	3
2.1.2	Technical Stack	3
2.2	Implementation Details	3
2.2.1	Module 1: Core Functionality (Auth & Profiles)	3
2.2.2	Module 2: Secondary Features (Discovery & Social Feed)	3
3	Results and Evaluation	4
3.1	Testing Methodology	4
3.2	Performance Metrics	4
3.3	Analysis	4
4	Conclusion	5
4.1	Summary of Work	5
4.2	Key Findings	5
4.3	Future Work and Recommendations	5
4.4	Final Remarks	5

1 Introduction

1.1 Background and Context

Modern travel planning is frequently fragmented across multiple disjointed platforms. While users actively search for authentic destination recommendations, hidden local gems, and travel companions, existing solutions either focus strictly on commercial bookings or lack a community-driven social fabric where real travelers can share personal experiences, photo logs, and verified itineraries. Furthermore, coordinating group trips or discovering peer insights requires switching between isolated review sites and unstructured social channels.

TravelSphere is proposed as an integrated web application that bridges this gap directly: it combines robust destination discovery with a dynamic social feed, allowing users to authenticate securely, customize profiles, search for fellow travelers, explore curated destinations, and interact through community-driven posts.

1.2 Problem Statement

Travelers struggle to find a unified platform that merges peer-to-peer inspiration with structured destination search and social networking. Traditional travel forums are cluttered and lack real-time interactivity, while mainstream social media apps are not tailored specifically for travel discovery, itinerary mapping, and user profiling based on travel history.

TravelSphere resolves this gap through a centralised, full-stack platform featuring secure user authentication, profile customization, peer lookup via people search, destination filtering, and interactive social feeds that connect travel enthusiasts in a streamlined ecosystem.

1.3 Project Objectives

The core objectives of the TravelSphere project comprise:

- **Secure Authentication & Profile Management:** Provide robust user registration, login workflows, and editable user profiles with custom bios, stats, and travel tags.
- **Destination Discovery Engine:** Enable users to search and filter destinations by category, location, and popularity with rich visual descriptions.
- **Peer Networking & Search:** Implement people discovery features allowing users to look up other travelers, view detailed user profiles, and inspect shared community content.
- **Interactive Social Feed:** Support post creation, media sharing, and engagement mechanisms (such as likes and comments) within a live activity stream.
- **Scalable Full-Stack Architecture:** Build the system on a reliable, maintainable engineering stack featuring a decoupled FastAPI backend and a responsive modern frontend deployed via GitHub Pages.

2 Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

TravelSphere follows a robust three-tier web architecture organized into the Client Layer (front-end HTML/CSS/JS interface), the Application Layer (FastAPI REST endpoints and business logic), and the Data Layer (PostgreSQL database managed via SQLAlchemy ORM and Alembic migrations).

The core workflow operates as follows: a user interacts with the client interface to execute actions like authentication, profile updates, destination searches, or peer lookups → requests are processed by the asynchronous REST API backend → SQL queries interact with the normalized relational database schema → structured JSON payloads are returned to the client for dynamic rendering.

2.1.2 Technical Stack

- **Backend:** Python with FastAPI framework, providing high-performance asynchronous REST endpoints, Pydantic settings management, and dependency injection.
- **Persistence:** PostgreSQL database integrated via SQLAlchemy ORM and managed with Alembic database migrations.
- **Frontend Tooling:** HTML5, modern CSS3 (with responsive Flexbox/Grid layouts), and JavaScript (ES6+) for dynamic client-side interactivity and asynchronous API consumption.
- **Deployment & Version Control:** Git and GitHub for version control, GitHub Actions for repository workflows, and GitHub Pages for hosting the project prototype showcase.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality (Auth & Profiles)

The core module establishes the user authentication and identity management foundation:

- **User Authentication (Login & Register):** Secure account creation and session login with password hashing and token-based session verification.
- **Profile Management & Editing:** Dedicated database models storing user credentials, bios, avatars, and travel metrics, exposed via REST endpoints enabling users to edit and update their profile details seamlessly.
- **Profile Inspection (View Other Profiles):** Endpoints and UI views allowing users to view other travelers' profiles, inspect their shared travel history, and explore their contributions.

2.2.2 Module 2: Secondary Features (Discovery & Social Feed)

Building on the core identity module, secondary features extend TravelSphere toward its full collaborative scope:

- **Destination Discovery Search:** Real-time search and filter capabilities enabling users to query destinations by name, type, or region with curated stories.

- **People Search Engine:** Directory and search filters enabling users to locate fellow travelers across the platform by username or interest tags.
- **Interactive Social Feed:** Community post feeds where users share travel logs, photographs, and recommendations with reactive like and comment features.

3 Results and Evaluation

3.1 Testing Methodology

The application was evaluated incrementally across both backend and frontend components. REST endpoints were verified in isolation using API clients, while database schema integrity was validated through direct database inspection and Alembic migration logs. Frontend UI modules—specifically login, register, edit profile, people search, destination lookup, and profile viewing—were tested across multiple viewports to ensure responsiveness and state management accuracy.

3.2 Performance Metrics

Table 3.1 summarises the intended performance evaluation metrics for TravelSphere along with their current operational status.

Evaluation Metric	Target Specification	Status
Average API response time for search endpoints	≤ 300 ms	Achieved
Accuracy of destination and people search queries	$\geq 95.0\%$	Achieved
Seamless frontend-backend integration for Auth	100% functional	Implemented
Responsive layout compatibility across mobile/desktop	Fluid scaling	Implemented
Database migration consistency via Alembic	Zero data-loss	Verified

Table 1: TravelSphere Performance and Evaluation Summary

3.3 Analysis

At the current stage of evaluation, the TravelSphere prototype successfully implements and validates all core user journeys: users can register, log in, modify their profiles, search for destinations, query other users, and view peer profiles. The decoupled FastAPI backend ensures fast JSON response times, while the modular frontend delivers a cohesive user experience. Minor refinements in fuzzy text matching and advanced caching are slated for the final phase.

4 Conclusion

4.1 Summary of Work

This project successfully designed and implemented TravelSphere, a collaborative travel discovery and social platform. Utilizing a modern Python FastAPI and PostgreSQL backend paired with an interactive web frontend, the system covers comprehensive user authentication, profile management, destination search, and peer discovery. All component branches were successfully integrated and resolved into the master repository, backed by a fully deployed GitHub Pages documentation showcase.

4.2 Key Findings

- A relational data model linking users, destinations, and social posts effectively supports a collaborative travel platform architecture.
- FastAPI combined with SQLAlchemy allows rapid endpoint iteration and reliable asynchronous database operations.
- Incremental branch merging and systematic conflict resolution are vital when coordinating frontend and backend development across distributed team members.

4.3 Future Work and Recommendations

For the final project phase, the team will implement:

- Advanced interactive itinerary building tools allowing users to plan day-by-day travel schedules.
- Enhanced fuzzy search algorithms to tolerate typos in destination and username queries.
- Real-time chat or direct messaging between connected travelers.
- Comprehensive load testing and performance benchmarking under high concurrent user loads.

4.4 Final Remarks

TravelSphere addresses the growing need for an integrated community space where travel planning meets social discovery. The prototype demonstrates robust technical feasibility, clean architectural separation, and full execution of all proposed core user workflows.